بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS-214 Computer Architecture and Organization

20 - Addressing

Introduction

- MIPS jump instruction has the simplest addressing
  - It uses the final MIPS instruction format, called the *J-type*
- Branch instruction
  - Must specify branch address

Jump Addressing

- Consists of 6 bits for the operation field (opcode)
- Rest of the bits for the address field
- For example,

```
j 10000 # go to location 10000
```

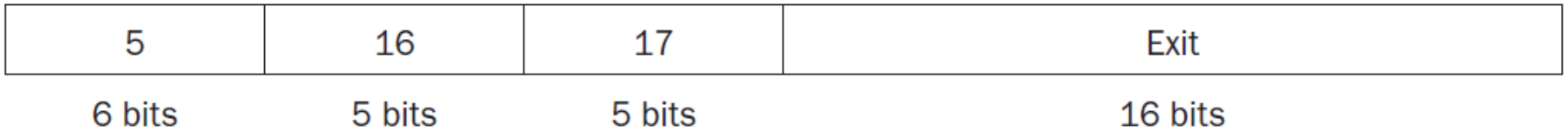
- Could be assembled into this format, where value of jump opcode is 2 and jump address is 10000

Branch Addressing

- Unlike jump instruction, conditional branch instruction must specify two operands in addition to the branch address
- For example,

```
bne $s0,$s1,Exit # go to Exit if $s0 ≠ $s1
```

- Assembled into this instruction, leaving only 16 bits for the branch address:



Address Size

- If addresses of program had to fit in this 16-bit field, it would mean that no program could be bigger than 2^{16}
- Too small to be a realistic option today

Address Size

- Alternative would be to specify a register that would always be added to branch address, so that a branch instruction would calculate the following

  Program counter = Register + Branch address

- Sum allows program to be as large as 2^{32} and still be able to use conditional branches, solving branch address size problem
  - Which register?
    - Answer comes from seeing how conditional branches are used

Address Size

- Conditional branches are found in loops and in *if* statements
  - so they tend to branch to a nearby instruction
- Since PC contains address of current instruction, we can branch within $\pm 2^{15}$ words of the current instruction if we use the PC as the register to be added to the address
- Almost all loops and *if* statements are much smaller than 2^{16} words, so PC is ideal choice
- This form of branch addressing is called PC-relative addressing

PC-relative Addressing

- It is convenient for HW to increment the PC early to point to the next instruction
- Hence, MIPS address is relative to address of the following instruction (PC + 4) as opposed to current instruction (PC)
- MIPS uses PC-relative addressing for all conditional branches,
  - Because destination of these instructions is likely to be close to branch

PC-relative Addressing

- On the other hand, `jal` instructions invoke procedures that have no reason to be near the call, so they normally use other forms of addressing
- Hence, MIPS architecture offers long addresses for procedure calls by using J-type format for both `j` and `jal` instructions

PC-relative Addressing

- Since all MIPS instructions are 4 bytes long, MIPS stretches the distance of branch by having PC-relative addressing refer to the number of *words* to the next instruction instead of the number of bytes
- Thus, 16-bit field can branch four times as far by interpreting the field as a relative word address rather than as a relative byte address
- Similarly, 26-bit field in jump instructions is also a word address, meaning that it represents a 28-bit byte address

PC-relative Addressing

- Since PC is 32 bits, 4 bits must come from somewhere else for jumps
- MIPS jump instruction replaces only the lower 28 bits of the PC, leaving the upper 4 bits of the PC unchanged

Example

- A *while* loop is compiled into the next MIPS assembler code. If the loop is placed starting at location 80000 in memory, what is the MIPS machine code for this loop?

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

Example (cont.)

- The assembled instructions, starting at address 80000:

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

80000

0	0	19	9	2	0
0	9	22	9	0	32
35	9	8	0		
5	8	21	?		
8	19	19	1		
2	?				

Example (cont.)

- MIPS instructions have byte addresses, so addresses of sequential words differ by 4, number of bytes in a word

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

80000	0	0	19	9	2	0
	0	9	22	9	0	32
	35	9	8	0		
	5	8	21	?		
	8	19	19	1		
	2	?				

Example (cont.)

- MIPS instructions have byte addresses, so addresses of sequential words differ by 4, number of bytes in a word

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

80000	0	0	19	9	2	0
80004	0	9	22	9	0	32
80008	35	9	8	0		
80012	5	8	21	?		
80016	8	19	19	1		
80020	2	?				
80024						

Example (cont.)

- bne instruction adds 2 words (8 bytes) to address of *following* instruction (80016), branch destination (8 + 80016 = 80024)

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

80000	0	0	19	9	2	0
80004	0	9	22	9	0	32
80008	35	9	8	0		
80012	5	8	21	2		
80016	8	19	19	1		
80020	2	?				
80024						

Example (cont.)

- jump instruction does use address (80000 = 20000 × 4), corresponding to label Loop

```
Loop: sll  $t1, $s3, 2
      add  $t1, $t1, $s6
      lw   $t0, 0($t1)
      bne  $t0, $s5, Exit
      addi $s3, $s3, 1
      j    Loop
Exit: …
```

80000	0	0	19	9	2	0
80004	0	9	22	9	0	32
80008	35	9	8	0		
80012	5	8	21	2		
80016	8	19	19	1		
80020	2	20000				
80024						

Branching Far Away

- Most conditional branches are to a nearby location
- But occasionally they branch far away
  - Farther than can be represented in 16 bits of conditional branch instruction
- Assembler comes to rescue
  - Inserts unconditional jump to branch target, and
  - Inverts condition so that branch decides whether to skip jump

Example

- Given a branch on register $s0 being equal to register $s1

```
beq $s0, $s1, L1
```

- Replace it by a pair of instructions that offers a much greater branching distance.

Example (cont.)

- Given a branch on register $s0 being equal to register $s1

```
beq $s0, $s1, L1
```

- Replace it by a pair of instructions that offers a much greater branching distance
- These instructions replace short-address conditional branch:

```
bne $s0, $s1, L2
j   L1
L2:
```

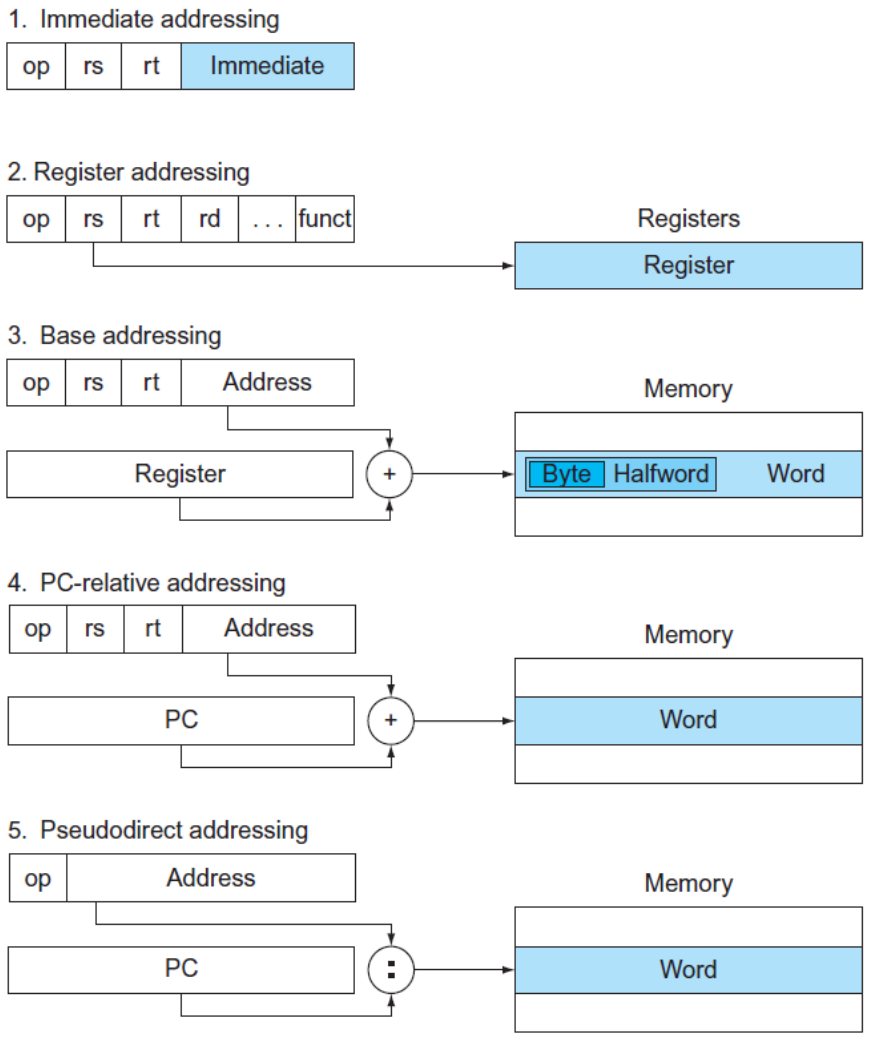
Addressing Modes Summary

- Multiple forms of addressing are generically called addressing modes
- Shows how operands are identified for each addressing mode
- Five MIPS addressing modes

Addressing Modes Summary

1. *Immediate addressing*, operand is constant within instruction
2. *Register addressing*, operand is register
3. *Base* or *displacement addressing*, operand is at memory location whose address is sum of a register and a constant in instruction
4. *PC-relative addressing*, branch address is sum of PC and a constant in instruction
5. *Pseudodirect addressing*, jump address is 26 bits of the instruction concatenated with upper bits of the PC

Addressing Modes Summary



64-bit Addresses

- Although we show 32-bit addresses, nearly all microprocessors (including MIPS) have 64-bit address extensions
- Extensions were in response to the needs of SW for larger programs
- Process of extension allows architectures to expand in such a way that is able to move SW compatibly upward to the next generation of architecture